

FastAPI Learning Journal

A complete beginner-to-pro reference — concepts, purposes, benefits, and interview answers. Learner: convoi.ai@gmail.com Started: 2026-05-18

Table of Contents

- [Foundation Concepts](#)
 - [Lesson 1 — HTTP Status Codes](#)
 - [Lesson 2 — Response Models](#)
 - [Lesson 3 — Input Validation](#)
 - [Lesson 4 — Polishing the Docs Page](#)
 - [Lesson 5 — APIRouter \(Multi-file Structure\)](#)
 - [Lesson 6 — Dependency Injection](#)
 - [Lesson 7 — async and await](#)
 - [Lesson 8 — Middleware and CORS](#)
 - [Lesson 9 — SQLAlchemy and SQLite \(Real Database\)](#)
 - [Lesson 10 — Authentication with JWT](#)
 - [Lesson 11 — Testing with pytest](#)
 - [Lesson 12 — Project Structure, Docker and Deployment](#)
 - [Terminal Commands Cheat Sheet](#)
 - [Interview Questions](#)
 - [Progress Tracker](#)
-

Foundation Concepts

The FastAPI app

The central "brain" of your API. Every URL, rule, and endpoint attaches to it.

- **Purpose:** Route incoming requests to the right function.
- **Benefit:** One single object that manages your entire API.

Pydantic models

Classes that describe the shape of data (fields and their types).

- **Purpose:** Automatically validate incoming JSON data.
- **Benefit:** You never have to write manual type checks. Bad data is rejected before your code runs.

HTTP verbs (decorators)

Words like GET, POST, PUT, DELETE that describe the intent of a request.

- **GET** → read data
- **POST** → create new data

- **PUT** → replace existing data
- **DELETE** → remove data
- **Purpose:** A universal convention for what a request wants to do.
- **Benefit:** Any client (browser, mobile app, third-party service) knows how to talk to your API.

Path parameters

Variables embedded inside the URL, like the ID in `/items/5`.

- **Purpose:** Identify a specific resource.
- **Benefit:** FastAPI auto-converts the type and validates it.

Query parameters

Optional extras after the `"?"` in a URL, used for filters, search, and pagination.

- **Purpose:** Modify how data is fetched, not which resource.
- **Benefit:** Clean way to pass optional information.

HTTPException

The FastAPI-native way to send an error response.

- **Purpose:** Stop the function and return a proper HTTP error.
- **Benefit:** Clean, standardized error handling — no manual error building.

The request lifecycle

1. Request arrives at Uvicorn
2. Uvicorn hands it to the FastAPI app
3. FastAPI matches URL + method to the right function
4. Pydantic validates the incoming data
5. Your function runs
6. FastAPI shapes the return value into JSON
7. Response is sent back to the client

- **Benefit:** You only write step 5 — FastAPI does the rest.

Lesson 1 — HTTP Status Codes

What they are

A three-digit number sent with every HTTP response, telling the client what happened.

The 5 families (by first digit)

- **1xx** — Informational (rarely used)
- **2xx** — Success
- **3xx** — Redirection
- **4xx** — Client error (the user did something wrong)

- **5xx** — Server error (your code broke)

Common codes to know

Code	Name	When
200	OK	General success with a response body
201	Created	A new resource was created (POST)
204	No Content	Success but no body to return (often DELETE)
400	Bad Request	Malformed request
401	Unauthorized	Not logged in
403	Forbidden	Logged in but not allowed
404	Not Found	Resource doesn't exist
422	Unprocessable Entity	Validation failed (Pydantic uses this)
500	Internal Server Error	Your code crashed

Purpose

Communicate the outcome of a request in a way every client can understand instantly.

Benefits

- **Standardized communication** — every dev, mobile app, and browser understands them
- **Enables smart clients** — frontends can react differently based on the code
- **Professional signaling** — using 201 instead of 200 shows your API is thoughtfully built
- **Better debugging** — the code alone tells you what type of problem occurred

Lesson 2 — Response Models

The problem

Pydantic controls what data comes IN, but nothing controls what goes OUT. This leads to:

- Inconsistent response shapes
- Accidental leaks of sensitive fields (like passwords)
- Poor auto-documentation of the response

The solution

Create two separate models — one for what the client sends (Input model), one for what the server returns (Output model).

The naming convention

Model	Job
ItemIn / ItemCreate	Data the client sends to create
ItemUpdate	Data the client sends to update
ItemOut / ItemResponse	Data the server sends back
ItemInDB	Full database shape with internal fields

Purpose

Give the server explicit control over what the response looks like.

Benefits

- **Security** — sensitive fields (passwords, tokens) can never accidentally leak
- **Consistency** — every response has the same predictable shape
- **Better docs** — Swagger shows the exact response schema
- **Refactor safety** — changing internal storage doesn't break the API contract
- **Separation of concerns** — input logic and output logic are cleanly split

Lesson 3 — Input Validation

The problem

Type hints alone accept any value of that type. An `int` could be `-5` or `999999999`. A `str` could be empty or a million characters long. This is dangerous.

The solution

Add explicit constraints using `Query()` for query parameters and `Path()` for path parameters.

Constraint types

For numbers:

- **gt** — greater than
- **ge** — greater than or equal
- **lt** — less than
- **le** — less than or equal

For strings:

- **min_length** — minimum characters
- **max_length** — maximum characters
- **pattern** — a regex the string must match

Universal:

- **default** — the default value if not provided
- **description** — appears in the /docs page

The pagination pattern (industry standard)

- **skip** — how many items to jump over
- **limit** — how many items to return

Every "Load More" or "Next Page" button on the internet is built on this pattern.

Purpose

Reject bad input at the door, before your function ever runs.

Benefits

- **Security** — no negative IDs, no ridiculous limits, no injection-length payloads
 - **Cleaner code** — zero manual validation inside your endpoints
 - **Automatic error messages** — Pydantic returns clean, standardized 422 responses
 - **Documentation as a byproduct** — constraints show up automatically in /docs
 - **Fail fast** — problems caught at the boundary, not deep in business logic
-

Lesson 4 — Polishing the Docs Page

What this lesson is

Zero new logic. One hundred percent presentation. It's about making the auto-generated /docs page look and read like a real product.

The five polish tools

- **title, description, version** on the FastAPI app — the top banner
- **tags** — groups endpoints into colored sections
- **summary** — a one-line description next to the URL
- **description or docstring** — the longer explanation
- **response_description** — labels what the successful response means
- **responses={404: {...}}** — officially documents error cases

The two auto-generated docs pages

- **/docs** (Swagger UI) — interactive, great for testing
- **/redoc** (ReDoc) — polished, great for public-facing documentation

Purpose

Communicate clearly with anyone who uses your API — teammates, frontend devs, mobile devs, future-you.

Benefits

- **Team velocity** — frontend and mobile devs stop asking "what does this endpoint do"
- **Onboarding** — new team members understand your API in minutes
- **Professional impression** — a polished /docs page signals a serious, mature codebase
- **Fewer bugs** — clarity prevents miscommunication about what data means

- **Self-documenting code** — docstrings serve both code and API documentation
 - **Free feature** — you get it just by adding a few keyword arguments
-

Lesson 5 — APIRouter (Multi-file Structure)

The problem

A single-file app works for six endpoints. It becomes a monster with sixty. Merge conflicts explode when multiple people edit the same file. Finding anything takes forever.

The solution

Split endpoints by topic into their own files. Each topic file uses **APIRouter** instead of the main app.

The canonical project structure

- **app.py** — main file, just plugs routers in
- **models.py** — all Pydantic models
- **storage.py** — the fake database (or real database session later)
- **routers/** — folder holding one file per topic
- **routers/init.py** — empty file marking the folder as a Python package
- **routers/items.py, users.py, orders.py** — actual endpoint files

The router's superpower — prefix and tags

When creating a router, you can set a URL prefix and a set of tags. Every endpoint in that router automatically inherits them. This means you don't repeat `"/items"` or the `"Items"` tag on every single endpoint.

Shared state trick

Python imports a module only once. Variables at the module level (like the fake items list) are shared across every file that imports them — everyone sees the same data.

Purpose

Keep the codebase modular, navigable, and scalable as it grows.

Benefits

- **Scalability** — supports hundreds of endpoints without becoming unmanageable
 - **Team collaboration** — multiple devs work on different routers with zero conflicts
 - **Feature isolation** — adding a new topic (users, orders) doesn't touch existing files
 - **Easier testing** — you can test one router without loading the entire app
 - **Faster navigation** — you always know exactly which file to open
 - **Real-world pattern** — every production FastAPI codebase looks like this
-

Lesson 6 — Dependency Injection

One-sentence definition

A helper function runs *before* your endpoint, and its result is handed to the endpoint automatically.

The kitchen analogy

- Your endpoint is the chef
- Dependencies are the kitchen helpers
- Helpers do the prep work first — wash vegetables, prep the bun
- Chef just cooks — helpers handled the prep

How it works

FastAPI sees that your endpoint is asking for a value from a helper function. It:

1. Runs the helper first
2. Passes the result to your endpoint
3. Handles any validation and errors along the way

Three common patterns

- **Function dependency** — the most common form; run a helper, use its result
- **Nested dependencies** — dependencies can themselves depend on other dependencies
- **Side-effect only** — run a check (like verifying an API key) and discard the return value

Real-world uses

- **Authentication** — "get the currently logged-in user"
- **Database sessions** — open and close a database connection
- **Shared query parameters** — pagination values used across many endpoints
- **Permission checks** — verify admin role before allowing access

The buzzword to know

Inversion of Control — the endpoint doesn't fetch what it needs; FastAPI provides it. This is a fundamental software engineering pattern.

Purpose

Eliminate duplicated logic and make endpoints focused on their unique work only.

Benefits

- **DRY code** — write shared logic once, reuse everywhere
- **Testability** — dependencies can be overridden in tests, making them ideal for mocking
- **Validation propagation** — dependencies bring their own validation rules along
- **Auto-documentation** — dependency parameters appear in /docs automatically
- **Composability** — dependencies can nest, enabling complex flows cleanly
- **Focused endpoints** — each endpoint contains only its unique business logic
- **Separation of concerns** — auth, DB, validation each live in their own dependency
- **Consistency** — no risk of one endpoint doing an auth check differently from another

Lesson 7 — async and await

One-sentence idea

`async` lets your server handle other requests while one is waiting for something slow (like a database or an external API call), instead of sitting idle.

The kitchen analogy

A synchronous chef puts water to boil and stands there waiting for 10 minutes. An async chef puts the water on, then chops vegetables, makes sauce, and sets the table while it heats up. Same chef, same stove — but the async chef uses waiting time productively.

When to use async

- Database queries
- External API calls
- File reads and writes
- Any I/O operation where you're waiting on something outside Python

When NOT to use async

- Pure CPU work like loops and math — there's no waiting, so nothing to gain
- Anywhere you'd have to call blocking code inside an async function

The syntax difference

- Regular function: `def`
- Async function: `async def`
- Inside an async function, use `await` in front of slow operations

FastAPI supports both

You can write endpoints with `def` or `async def`. FastAPI handles them correctly either way. Sync endpoints run in a thread pool; async endpoints run directly on the event loop.

The big warning

Never do slow blocking work inside an `async def` without `await`. That freezes the entire server — all other requests stall.

Purpose

Handle many simultaneous requests efficiently without needing many workers or threads.

Benefits

- **Massive scalability** — one server can handle thousands of concurrent requests
- **Better resource usage** — no idle threads waiting on slow operations
- **Lower cost** — fewer servers needed to serve the same traffic
- **Native support in FastAPI** — the framework was built for it

- **Modern Python** — async is the future of I/O-heavy Python code

When it'll matter for us

Lessons 9 (database) and 10 (auth). For in-memory work like ours today, async brings no benefit — but it's the mental model you need before we hit real I/O.

Lesson 8 — Middleware and CORS

Middleware — one-sentence idea

Code that runs on every request, before it reaches your endpoint and after your endpoint sends a response.

The security guard analogy

Every visitor to a building passes through the guard on the way in and on the way out. The guard can inspect them, log them, or block them entirely. Middleware plays that role for HTTP requests.

The request flow with middleware

1. Request arrives
2. Middleware runs (can inspect, modify, or reject)
3. Endpoint runs
4. Middleware runs again (can inspect or modify the response)
5. Response goes out

Common uses of middleware

- Logging every request
- Timing endpoint performance
- Authentication checks before any endpoint runs
- CORS handling
- Attaching request IDs for tracing
- Automatic response compression (gzip)

Purpose of middleware

Give you a place to handle cross-cutting concerns once, without adding the same code to every endpoint.

Benefits of middleware

- **Cross-cutting concerns solved centrally** — logging, timing, auth, etc.
- **DRY** — no duplication across endpoints
- **Framework-level enforcement** — nothing can slip past it
- **Reusable** — same middleware across apps
- **Debugging superpower** — trace every request without touching endpoint code

CORS — one-sentence idea

A browser security rule that blocks JavaScript from calling your API when it's on a different domain, unless your API explicitly allows it.

The problem in plain English

Your frontend runs on one origin (like localhost:3000). Your backend runs on another (localhost:8000). Different origins. The browser blocks the frontend from calling your API by default — for security reasons.

Why the browser enforces this

Without CORS, any malicious website could quietly call APIs on behalf of a logged-in user using their cookies. CORS makes the API explicitly agree to be called from a given origin.

The FastAPI solution

FastAPI ships with a built-in CORSMiddleware. You configure which origins, methods, and headers are allowed, and the browser is satisfied.

The 3 CORS settings you'll use daily

- **allow_origins** — list of frontend URLs allowed to call your API
- **allow_methods** — which HTTP verbs are permitted
- **allow_headers** — which headers frontends can send

Purpose of CORS

Allow legitimate browser-based frontends to call your API safely, while blocking unauthorized origins.

Benefits of CORS

- **Enables real frontends** — without it, no React/Vue/Angular app can talk to your API
- **Security by default** — you explicitly whitelist who can call you
- **Fine-grained control** — per-origin, per-method, per-header
- **Prevents attacks** — protects users from malicious cross-site requests
- **Standard practice** — every production API deals with CORS

When CORS actually matters

The moment you connect a browser-based frontend to your API. Day one of any full-stack project.

Lesson 9 — SQLAlchemy and SQLite (Real Database)

One-sentence idea

Stop storing data in a Python list. Store it in a real database that persists forever, handles millions of rows, and can be queried with SQL.

The problem with in-memory storage

A Python list lives in RAM. Stop the server and the data is gone. Two servers each have their own list. Not usable at real scale.

What SQLAlchemy is

The most popular ORM (Object-Relational Mapper) in Python. It lets you work with database rows as if they were Python objects, without writing raw SQL. When you write "give me item 5", SQLAlchemy translates it into a proper SQL query.

What SQLite is

A tiny file-based database. The whole thing lives in a single .db file on disk. No server to install, no configuration. Perfect for learning and small apps. Later you can swap it for PostgreSQL or MySQL with almost no code change.

The four building blocks

- **Engine** — the low-level database connection
- **Session** — a short conversation with the database (add, query, commit)
- **Model** — a Python class that maps to a database table
- **Migration** — the recipe for creating or updating tables

Two kinds of models — kept separate

- **Pydantic models** — describe what the API accepts and returns
- **SQLAlchemy models** — describe what the database stores

They stay separate because they have different jobs. Pydantic protects the API boundary; SQLAlchemy handles persistence.

The new project shape

- `database.py` — engine + session setup
- `db_models.py` — SQLAlchemy models (the tables)
- `models.py` — Pydantic models remain for validation
- `dependencies.py` — a `get_db` dependency that opens and closes sessions per request

Purpose

Replace fake in-memory storage with real persistent, queryable, scalable data.

Benefits

- **Data survives restarts** — the entire point of a database
- **Handles scale** — millions of rows with indexed queries
- **Relationships** — join items to users, orders, categories, etc.
- **Transactions** — all-or-nothing operations (banking-safe)
- **Portability** — swap SQLite for PostgreSQL by changing one config line
- **Concurrency** — safe when many users write at the same time
- **Backups and replication** — real databases can be backed up, replicated, restored

- **Query power** — filtering, sorting, aggregations, joins all built in

The mental shift

This is the lesson where the API stops being a toy and starts being production-capable. Everything after this — auth, testing, deployment — assumes you have real persistence.

Lesson 10 — Authentication with JWT

One-sentence idea

After the user logs in, give them a signed "ticket" (a token). They send it with every future request to prove who they are.

The problem

Your API has no idea who's calling it. Anyone can create or delete data. No accounts, no permissions, no identity.

What JWT is

JSON Web Token. A long string with three parts separated by dots:

1. **Header** — describes the algorithm used
2. **Payload** — data about the user (user id, role, expiry)
3. **Signature** — a cryptographic seal that proves the token was not tampered with

The full authentication flow

1. User sends email + password to a login endpoint
2. Server verifies the password against the hashed version in the database
3. Server creates a JWT containing the user's ID and sends it back
4. Client stores the token (typically in memory or localStorage)
5. Client sends the token in the Authorization header on every future request
6. Server verifies the signature and extracts the user info

The three tools you'll use in FastAPI

- **passlib** — safely hashes passwords using bcrypt or argon2
- **python-jose** — creates and verifies JWTs
- **OAuth2PasswordBearer** — FastAPI's helper for reading tokens from requests

The `get_current_user` dependency

This is where Lesson 6 (Dependency Injection) truly shines. You write one dependency that:

- Reads the token from the request
- Verifies its signature
- Extracts the user id
- Fetches the user from the database

- Returns the user object

Any endpoint that needs authentication just adds `user = Depends(get_current_user)` — one line, everywhere.

Critical security rules

- **Never store passwords in plain text** — always hash with bcrypt or argon2
- **Never put secrets in the payload** — the payload is readable by anyone
- **Always set expiration** — usually 15 minutes to 24 hours
- **Always use HTTPS in production** — otherwise tokens can be stolen
- **Keep the signing key secret** — anyone with it can forge tokens

Authentication vs Authorization

- **Authentication** = "who are you?" (login)
- **Authorization** = "what are you allowed to do?" (permissions/roles)

They are separate concepts. JWT commonly handles both — the payload carries identity (auth) and role information (permissions).

Purpose

Identify users, protect endpoints, and enforce fine-grained permissions.

Benefits

- **Stateless** — the server doesn't store session data; the token carries it
- **Scalable** — any server can verify the token; no shared session store required
- **Standard** — JWT is used by nearly every modern API
- **Fine-grained control** — role and permission data lives inside the token
- **Cross-platform** — works identically for web, iOS, Android, and CLI clients
- **Testable** — auth flows can be unit tested cleanly
- **Composable** — combined with FastAPI's Depends, protecting any endpoint is one line

Lesson 11 — Testing with pytest

One-sentence idea

Write small automated scripts that call your API and check the responses — so you catch bugs before your users do.

The problem manual testing solves poorly

Manually clicking around /docs works for a few endpoints. With 50 endpoints across multiple features, manual testing becomes slow, error-prone, and impossible to repeat every time you change code.

What pytest is

The most popular Python testing framework. You write functions whose names start with `test_` and `pytest` finds and runs them all with a single command. When an assertion fails, you know immediately what broke.

What TestClient is

A FastAPI helper that behaves like a real HTTP client but calls your app directly in memory. No need to run the actual server. Fast, reliable, and perfect for tests.

The AAA pattern (Arrange, Act, Assert)

Every good test has three phases:

- **Arrange** — set up the scenario (create data, log in a user)
- **Act** — make the HTTP call
- **Assert** — check that the response matches what you expected

This structure makes tests easy to read and reason about.

Fixtures — pytest's dependency injection

A fixture is reusable setup code. You define it once with a decorator and any test can request it by adding it as a parameter. Fresh databases, logged-in users, seed data — all common fixtures. The concept is very close to FastAPI's `Depends`.

Dependency overrides — the killer feature for FastAPI

Because FastAPI uses `Depends` everywhere (for the database session, for the current user, for permissions), tests can replace them with fakes. Swap the real database for an in-memory test database. Swap the real current-user check with a mock user. This lets you test any endpoint in complete isolation.

What good FastAPI tests cover

- Happy paths — normal successful calls
- Validation errors — bad input should return 422
- Not-found cases — missing resources should return 404
- Auth failures — no token should return 401
- Permission failures — wrong role should return 403
- Edge cases — empty lists, boundary values, duplicates

Unit vs integration tests

- **Unit tests** check one small piece in isolation (a single function)
- **Integration tests** check multiple pieces working together (endpoint + database)

Most FastAPI tests are integration tests because they exercise the request pipeline end-to-end.

Purpose

Catch bugs automatically before they reach users and keep the code safe to change.

Benefits

- **Confidence** — you can refactor without breaking things
 - **Speed** — hundreds of tests run in seconds
 - **Living documentation** — tests show exactly how endpoints should behave
 - **Regression prevention** — a fixed bug stays fixed
 - **Team scaling** — new devs can change code safely with a test safety net
 - **CI/CD ready** — tests run automatically on every commit and PR
 - **Better design** — code that is hard to test is usually badly designed; tests guide better architecture
 - **Deployment safety** — no broken code ships if the test suite fails
 - **Industry expectation** — no professional project ships without tests
-

Lesson 12 — Project Structure, Docker and Deployment

The final lesson

Getting your app off your laptop and onto the internet — professionally, safely, and repeatably.

Production project structure

- **app/** — all source code lives in a single package
- **app/main.py** — FastAPI app + include_router calls
- **app/models.py** — Pydantic models
- **app/db_models.py** — SQLAlchemy models
- **app/database.py** — engine + session setup
- **app/config.py** — settings loaded from environment variables
- **app/dependencies.py** — shared dependencies
- **app/security.py** — JWT and password hashing
- **app/routers/** — one file per feature
- **tests/** — tests live in their own folder
- **.env** — secrets, never committed
- **.gitignore** — must exclude .env and other secrets
- **requirements.txt** — pinned dependency versions
- **Dockerfile** — build recipe
- **README.md** — how to run the project

Rules of thumb

- Code stays inside a single **app/** package
- Tests live outside the app folder
- Config is loaded from environment variables, never hardcoded
- Secrets never live in code — they live in **.env** (which is gitignored)

Environment variables and pydantic-settings

The problem with hardcoding values like database URLs or JWT secrets is security and inflexibility. Storing them as environment variables keeps them out of code and lets different environments use different values.

FastAPI's companion library **pydantic-settings** loads env vars into a typed Settings model — same validation you already know from Pydantic.

Docker — the one-sentence idea

Docker packages your app plus its Python version plus its dependencies into a single portable "box" that runs the same on any machine.

The classic problem Docker solves

"It works on my machine!" — the developer's nightmare. Different laptops have different Python versions and libraries. Docker eliminates that by shipping the exact environment along with the code.

Docker building blocks

- **Dockerfile** — the recipe for building your image
- **Image** — a frozen snapshot of your app plus its environment
- **Container** — a running instance of an image

A typical Dockerfile flow (in plain English)

1. Start from a small Python base image
2. Copy requirements.txt in
3. Install dependencies
4. Copy the source code in
5. Tell Docker what command to run when the container starts

Push the resulting image to a registry (Docker Hub, AWS ECR, GitHub Container Registry) — any server can pull and run it identically.

Deployment options

- **Render** or **Railway** — best for beginners, free tier, deploys from GitHub
- **Fly.io** — good for global edge deployment
- **AWS / GCP / Azure** — production at scale
- **DigitalOcean** — cheap and flexible VPS

For a first real project, Render or Railway is the sweet spot.

The production checklist

- Secrets live in env vars, not code
- `--reload` is off in production
- CORS origins are locked to your real frontend domain
- HTTPS is enabled
- Logging is configured
- Tests pass in CI before every deploy

Production process manager

In production, you don't run `uvicorn app:app --reload`. You run **Gunicorn with Uvicorn workers** — this gives you multiple worker processes for handling more concurrent traffic and stability.

CI/CD — one-sentence idea

Every time you push code to GitHub, a robot automatically runs your tests and — if they pass — deploys the new version. Tools include GitHub Actions, GitLab CI, and CircleCI.

Purpose

Get your app off your laptop, into a professional structure, containerized, and running on the public internet — safely, reliably, and repeatably.

Benefits

- **Consistency** — Docker ensures dev, staging, and production behave identically
- **Portability** — the same image runs on any cloud
- **Scalability** — spin up more containers to handle more traffic
- **Isolation** — your app doesn't fight other apps for system libraries
- **Fast onboarding** — new devs start the whole project with one command
- **Real-world skill** — Docker is a top-3 backend hiring requirement
- **Security** — secrets isolated, HTTPS enforced, CORS locked down
- **Automation** — CI/CD means shipping code is a git push, not a nervous ritual
- **Rollback** — bad deploy? Rebuild the previous image for instant recovery
- **Multi-environment** — same code deploys cleanly to dev, staging, and production

Terminal Commands Cheat Sheet

Command	Purpose
<code>python --version</code>	Check Python version
<code>pip install fastapi</code>	Install FastAPI framework
<code>pip install "uvicorn[standard]"</code>	Install Uvicorn server
<code>uvicorn app:app --reload</code>	Start server (auto-reloads on save)
<code>Ctrl + C</code>	Stop the server
<code>pip freeze > requirements.txt</code>	Save dependencies to a file
<code>pip install -r requirements.txt</code>	Install from a requirements file
<code>python -m venv venv</code>	Create a virtual environment
<code>.\venv\Scripts\Activate.ps1</code>	Activate venv (Windows PowerShell)
<code>deactivate</code>	Leave the venv

The "app:app" decoded

`filename : variable_name` → the `app` variable inside the file `app.py`.

Important URLs

URL	What's there
http://127.0.0.1:8000/	Root endpoint
http://127.0.0.1:8000/docs	Swagger UI (interactive tester)
http://127.0.0.1:8000/redoc	ReDoc (polished docs)

Interview Questions

Q1 — What is FastAPI?

A modern, high-performance Python web framework for building APIs. It uses Python type hints for automatic data validation, serialization, and interactive documentation. Built on Starlette for async and Pydantic for validation.

Q2 — Why FastAPI over Flask or Django?

Automatic request validation via Pydantic. Auto-generated OpenAPI and Swagger docs. Native async and await support. One of the fastest Python frameworks. Modern type hints as first-class citizens.

Q3 — What is Pydantic and why do we use it?

Pydantic is a data validation library. In FastAPI, we use it to define the shape of request and response bodies. It automatically validates incoming JSON — bad data is rejected before our endpoint runs, and clean data is guaranteed inside our function.

Q4 — Difference between Path and Query parameters?

Path parameters are part of the URL itself and identify a specific resource. Query parameters come after the question mark and are used for optional modifiers like filters, search, and pagination.

Q5 — What's the difference between 200, 201, and 204?

200 is general success with a body. 201 means a new resource was created — typical for POST. 204 means success with no response body — typical for DELETE.

Q6 — What does 422 mean in FastAPI?

Unprocessable Entity. The request was well-formed but failed validation. FastAPI returns 422 when incoming data doesn't match the expected Pydantic schema.

Q7 — Why have separate ItemIn and ItemOut models?

Different jobs. The client doesn't send an item_id when creating (the server assigns it), but the client should receive it in the response. Separate output models also prevent leaks of internal fields like passwords, and they produce cleaner documentation.

Q8 — What does response_model do?

It tells FastAPI to shape the outgoing response to match the given model. Extra fields are stripped, the response schema is auto-documented in Swagger, and internal data can never accidentally leak out.

Q9 — What is APIRouter and why use it?

APIRouter is a mini-FastAPI application for organizing endpoints by topic. Each router lives in its own file, and the main app plugs them in with `include_router`. This keeps the codebase modular, easy to navigate, and prevents merge conflicts when many devs work on the same project.

Q10 — What is Dependency Injection in FastAPI?

A way to share reusable logic across endpoints. You write a helper function, and any endpoint declares its result as a parameter using `Depends`. FastAPI runs the dependency first, passes the result in, and handles validation and errors. It's used for auth, database sessions, pagination, and permission checks.

Q11 — What are the benefits of Dependency Injection?

DRY code — write once, reuse everywhere. Validation propagation — dependencies carry their own rules. Auto-documentation — dependency parameters appear in `/docs`. Testability — dependencies can be overridden in tests. Inversion of Control — endpoints don't fetch what they need; FastAPI provides it.

Q12 — What is Uvicorn?

An ASGI web server. FastAPI is the framework — the rules and tools. Uvicorn is the actual engine that listens on a port and feeds HTTP requests into the FastAPI app.

Q13 — What does `--reload` do?

Uvicorn watches your files and auto-restarts the server on any change. It's a development-only feature and should never be used in production.

Q14 — What does `HTTPException` do?

It's the FastAPI-native way to return an HTTP error. You raise it with a status code and detail, and FastAPI turns it into a proper JSON error response — cleaner than manually building error responses.

Q15 — What's the purpose of tags on endpoints?

Groups endpoints into sections in the `/docs` page. Purely for organization and readability — no effect on behavior. Essential when your API has many endpoints.

Q16 — What is inversion of control?

A design principle where control of when and how something runs is handed to the framework rather than being managed by your code. Dependency Injection is a common form of it. In FastAPI, your endpoint declares what it needs and FastAPI provides it, instead of the endpoint fetching it itself.

Q17 — What does auto-generated OpenAPI documentation mean?

FastAPI reads your code — the paths, models, validation rules, tags, and summaries — and produces a machine-readable OpenAPI schema. From that, the Swagger UI and ReDoc pages are built automatically,

giving you interactive documentation without writing any of it yourself.

Q18 — Why is Pydantic v2's `model_dump()` preferred over `dict()`?

`model_dump` is the current, official method in Pydantic v2 for converting a model into a plain dict. The older `dict` method is deprecated and will eventually be removed. Using `model_dump` means your code stays compatible with modern Pydantic.

Q19 — What is the difference between `def` and `async def` in FastAPI?

`def` runs synchronously — the request occupies a thread until it finishes. `async def` runs asynchronously — while awaiting an I/O operation, the server can process other requests. FastAPI supports both.

Q20 — When should you use `async` in FastAPI?

When your endpoint does I/O work — database queries, external API calls, file operations. Anywhere there's waiting for something outside Python.

Q21 — When should you NOT use `async`?

For pure CPU work like loops and math — `async` gives no benefit because there's nothing to wait on. Also avoid using `async def` if you're going to call blocking code inside without `await`.

Q22 — What's the benefit of `async` at scale?

One server can handle thousands of concurrent requests efficiently — you're not blocked while waiting on slow operations. This means fewer servers, lower cost, and higher throughput.

Q23 — What happens if you call blocking code inside an `async` function?

The event loop is frozen — every other request in the server stalls until that blocking call finishes. It defeats the purpose of `async` and can bring your API to its knees under load.

Q24 — What is middleware in FastAPI?

Code that runs on every request, before and after the endpoint. Used for logging, timing, auth checks, CORS, request IDs, and other cross-cutting concerns.

Q25 — What is CORS?

Cross-Origin Resource Sharing. A browser security mechanism that blocks JavaScript from calling APIs on a different origin unless the API explicitly allows it.

Q26 — Why does CORS exist?

To prevent malicious websites from calling APIs on behalf of a logged-in user without their consent. It's a browser-enforced protection.

Q27 — How do you enable CORS in FastAPI?

By adding `CORSMiddleware` from `fastapi.middleware.cors` and configuring the allowed origins, methods, and headers.

Q28 — What's the difference between CORS and authentication?

CORS controls which origins can make requests. Authentication controls which users can access data. They're independent — you can have both, either, or neither.

Q29 — What is a preflight request?

Before certain requests (like POST with JSON), browsers send an `OPTIONS` request first asking the server "am I allowed to do this?" That's the preflight. The CORS middleware handles it automatically.

Q30 — Can middleware modify the response?

Yes. Middleware sees both the request and the response, and can modify either. For example, adding a custom header to every response or logging response times.

Q31 — What is an ORM?

Object-Relational Mapper. It lets you work with database rows as Python objects instead of writing raw SQL. SQLAlchemy is the most popular Python ORM.

Q32 — Why use SQLAlchemy?

It's database-agnostic — the same code works with SQLite, PostgreSQL, MySQL. It provides a powerful query API, transaction support, and connection pooling.

Q33 — What is a SQLAlchemy session?

A short-lived conversation with the database. You add, query, and commit through it. In web apps, a session is typically opened at the start of each request and closed at the end.

Q34 — Why keep Pydantic and SQLAlchemy models separate?

Different responsibilities. Pydantic validates API input and output. SQLAlchemy defines database storage. Separating them keeps the API contract independent from the database schema.

Q35 — What's the difference between SQLite and PostgreSQL?

SQLite is file-based, single-user friendly, and requires no setup — great for learning and small apps. PostgreSQL is a full-featured server-based database with better concurrency, scaling, and advanced features — the standard choice for production.

Q36 — What is a database migration?

A version-controlled recipe for changing the database schema — adding tables, columns, indexes. Tools like Alembic manage migrations so schema changes can be applied and rolled back safely.

Q37 — What is JWT?

JSON Web Token. A signed token containing user information that clients send with each request to prove their identity.

Q38 — What are the three parts of a JWT?

Header (algorithm used), Payload (user data), Signature (proves the token was not tampered with).

Q39 — Why hash passwords instead of storing them?

So that if the database is stolen, attackers cannot recover the original passwords. Hashing is one-way — even the server can't reverse it.

Q40 — What's the difference between authentication and authorization?

Authentication is "who are you?" (login). Authorization is "what are you allowed to do?" (permissions and roles). They're independent but often go together.

Q41 — Why is JWT called stateless?

The server does not store any session data. The token itself carries the user info. Any server that can verify the signature can serve the request — great for horizontal scaling.

Q42 — What's the risk of not expiring tokens?

If a token is stolen or leaked, it works forever. Expiration limits the window of damage. Refresh tokens allow long-lived sessions safely.

Q43 — Where should JWTs be stored on the frontend?

It's a trade-off. `localStorage` is convenient but vulnerable to XSS. HTTP-only cookies are safer against XSS but require CSRF protection. No perfect answer — depends on the app's threat model.

Q44 — What does `OAuth2PasswordBearer` do in FastAPI?

It's a helper that reads a bearer token from the Authorization header of the request and makes it available to your dependency. It also integrates with the `/docs` page so you can log in interactively.

Q45 — How would you protect an endpoint with FastAPI?

Create a `get_current_user` dependency that verifies the token and returns the user. Then add `user = Depends(get_current_user)` to any endpoint that requires authentication.

Q46 — Why write tests?

To catch bugs early, prevent regressions, enable safe refactoring, and document expected behavior. Untested code is a liability that grows more expensive over time.

Q47 — What is pytest?

The most popular Python testing framework. It auto-discovers functions whose names start with `test_` and runs them with a single command.

Q48 — What is TestClient in FastAPI?

A helper that acts like an HTTP client but calls your FastAPI app directly in memory. No real server needed, so tests are fast and reliable.

Q49 — What is a pytest fixture?

Reusable setup code. Define it once with a decorator and request it from any test. Common examples: fresh databases, seed data, logged-in users. The concept is similar to FastAPI's Depends.

Q50 — What is dependency override in FastAPI tests?

Replacing a real dependency like `get_db` or `get_current_user` with a fake one during tests. Lets you isolate what you're testing without hitting the real database or authentication service.

Q51 — What is the AAA pattern in testing?

Arrange, Act, Assert. Set up the scenario, make the call, check the result. Standard structure that makes tests easy to read.

Q52 — What's the difference between unit tests and integration tests?

Unit tests check one function in isolation — fast and focused. Integration tests check multiple pieces working together, like an API endpoint plus its database. Most FastAPI tests are integration tests.

Q53 — What is a good test coverage target?

Typically 70–90% for real projects. But coverage percentage alone doesn't guarantee quality — well-designed tests that cover the important paths matter more than raw numbers.

Q54 — Why is dependency injection important for testing?

Because dependencies can be swapped out in tests. Without DI, you have to test with real databases, real auth, real external calls. With DI, you swap in fakes and test any layer in isolation.

Q55 — What is Docker?

A tool that packages an app and its dependencies into a portable container that runs identically on any machine. Solves the "works on my machine" problem.

Q56 — What's the difference between a Docker image and a container?

An image is a frozen snapshot of an app plus its environment. A container is a running instance of an image. Similar to the class-vs-object relationship.

Q57 — What is a Dockerfile?

A recipe describing how to build a Docker image — the base OS, dependencies, source code, and startup command.

Q58 — Why use environment variables for config?

To keep secrets and configuration out of code. Different environments (dev, staging, prod) can use different values without changing the code itself.

Q59 — Why not use `uvicorn --reload` in production?

It's a development-only feature. Auto-reload is expensive, unnecessary, and less stable. In production you use Gunicorn with Uvicorn workers for multiple processes and reliability.

Q60 — What is CI/CD?

Continuous Integration and Continuous Deployment. Every code push triggers automated tests, and passing code is automatically deployed. Standard modern development practice.

Q61 — What's the difference between Docker and a virtual environment?

A venv isolates only Python packages. Docker isolates the entire OS-level environment — Python version, system libraries, everything. Docker is heavier but far more portable.

Q62 — What is a container registry?

A place to store and share Docker images. Common ones are Docker Hub, AWS ECR, and GitHub Container Registry.

Q63 — How would you deploy a FastAPI app to production?

Options include Render or Railway for simple hosting, or Docker plus AWS/GCP/Azure for full control. Load config from env vars, enforce HTTPS, run behind Gunicorn with Uvicorn workers, and gate deploys on passing tests in CI.

Q64 — Why is pydantic-settings useful?

It loads environment variables into a typed Pydantic settings model, giving you the same validation and IDE autocomplete you get for API models. Prevents typos and missing config from silently breaking your app.

Q65 — How do you scale a FastAPI application?

Horizontally — run multiple Uvicorn workers behind Gunicorn, then run multiple containers behind a load balancer. Async endpoints, connection pooling in the DB, and caching are all common tools to increase throughput per instance.

Progress Tracker

Lesson	Topic	Status
1	HTTP Status Codes	Complete
2	Response Models	Complete
3	Input Validation	Complete
4	Docs Polish	Complete

Lesson	Topic	Status
5	APIRouter	Complete
6	Dependency Injection	Complete
7	async and await	Complete
8	Middleware and CORS	Complete
9	SQLAlchemy Database	Complete
10	Authentication and JWT	Complete
11	Testing with pytest	Complete
12	Project Structure and Deployment	Complete

Overall progress: about 90% to FastAPI Pro — all 12 lessons complete!

The final 10% comes from building your own real project — which is the next chapter of your journey.